

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(УНИВЕРСИТЕТ ИТМО)

ОТЧЁТ

по заданию 3 «Генераторы статических сайтов для публикации
результатов исследований»

Тема: стабилизация взгляда в VR, воспроизводимая публикация
результатов

Образовательная программа	Разработка ассистивных ИТ-решений (магистратура)
Группа	1.3
Выполнила	Кожевяткина Марина Алексеевна
Выполненные задания	исследовательское ТЗ, практическое РЗ, GitHub Action (задание повышенной сложности)

Санкт-Петербург
2026

Аннотация

Работа посвящена воспроизводимой публикации результатов исследования: расчёт по данным встроен в сборку статического сайта, сайт собирается и выкладывается конвейером CI без ручных действий. В качестве данных использована моя пилотная запись взгляда в VR-музее (8 сессий, 17 132 строки). Выполнены исследовательское задание ТЗ (обзор отечественных платформ CI/CD и хостингов), практическое задание РЗ (конвейер «данные → результат → сайт»), а также оформлен и опубликован собственный GitHub Action `rsync-ssh-deploy` для выкладки по SSH.

Сайт опубликован на GitHub Pages и Cloudflare Pages, сборка и выкладка выполняются в GitHub Actions. Все измерения (время сборки и развёртывания, вес страниц) приведены в таблицах, ошибки, найденные в работе, разобраны в разделе «Отладка».

Содержание

№	Раздел	Что внутри
1	Введение	цель, задачи, исходные данные, список репозиторий и сайтов
2	Ход работы	шаги задания, описание пайплайна по заданиям, базовый URL, проверка выкладки
3	РЗ. Конвейер данные → результат → сайт	практическое задание: расчёт по записи взгляда в сборке сайта
4	ТЗ. Отечественные CI/CD и хостинги	исследовательское задание: сравнение платформ, оценка миграции
5	GitHub Action rsync-ssh-deploy	собственный action: параметры, версии, тесты, использование
6	Измерения	время сборки и развёртывания, вес страниц
7	Отладка	реальные ошибки, гипотезы, проверки, решения, проваленный и успешный запуски
8	Заключение	рекомендуемый стек и ограничения
9	Ссылки, источники и лицензии	репозитории, сайты, источники, лицензии
A	Приложение: тексты пайплайнов	<code>deploy.yml</code> , <code>action.yml</code> , тестовый и релизный workflow

Метка версии этого сайта

Параметр	Значение
Коммит	fc452ee
Дата сборки (UTC)	2026-10-05 12:20:56
Версия набора данных	pilot-2026-09-16
SHA-256 набора данных	4a161632da0e2ed9...
Файлов / строк	8 / 17132
Версия скрипта	1.0

1. Введение

1.1. Цель и задачи

Цель работы — показать, как результаты исследования по записи взгляда в VR публикуются в виде статического сайта так, чтобы опубликованная страница всегда соответствовала версии данных и кода.

Задачи:

1. Выполнить исследовательское задание ТЗ: сравнить отечественные платформы CI/CD и варианты размещения статического сайта, оценить трудозатраты на перенос workflow.
2. Выполнить практическое задание РЗ: встроить расчёт по данным (таблицы, графики, метка версии) в сборку сайта, показать обновление при изменении данных и работу кэша.
3. Настроить конвейер GitHub Actions (проверка, сборка, выкладка на два хостинга, проверка опубликованной страницы).
4. Оформить и опубликовать собственный GitHub Action для выкладки по SSH с семантическим версионированием, лицензией и README.
5. Снять измерения (время сборки и развёртывания, вес страниц) и разобрать ошибки, возникшие в работе.

1.2. Исходные данные

Данные — моя пилотная запись из VR-музея, сделанная 16 сентября 2026 года: 8 CSV-файлов, 17 132 строки, 206 секунд записи. Подробное описание формата и особенностей данных — в разделе [4. РЗ](#).

Сырые записи взгляда относятся к персональным данным участников, поэтому они не покрываются лицензиями репозитория (см. [раздел 9](#)).

1.3. Созданные репозитории и опубликованные сайты

Что	Адрес
Репозиторий сайта (отчёт, конвейер, данные)	https://github.com/777werona-afk/vr-gaze-site
Репозиторий action <code>rsync-ssh-deploy</code>	https://github.com/777werona-afk/rsync-ssh-deploy
Сайт на GitHub Pages	https://777werona-afk.github.io/vr-gaze-site/
Сайт на втором хостинге (Cloudflare Pages)	https://vr-gaze-site.pages.dev

⚠ Отступление от условий

Условия просят опубликовать сайт на GitHub Pages и на отечественном хостинге. Отечественный хостинг в этой работе не разворачивался: второй площадкой выбран Cloudflare Pages, который к отечественным не относится. Причины выбора: бесплатный тариф без привязки банковской карты, выкладка одной командой с адресом `pages.dev` и HTTPS, а доступ к хостингу университета и платные российские сервисы требуют учётных данных и оплаты, которых у меня не было (подробнее — в [заключении](#)). Для перехода на отечественный хостинг подготовлено всё, кроме самого сервера: сравнение вариантов в разделе ТЗ и action `rsync-ssh-deploy`, которому достаточно SSH-доступа и ключа. Выкладку по SSH на постоянный сервер я не проверяла: проверка проведена на локальном `ssh` внутри раннера CI.

1.4. Структура отчёта

Отчёт опубликован самим сайтом в разделе «Ход работы» и связанных разделах. Нумерация разделов совпадает с оглавлением на титульной странице. Скриншоты и таблицы относятся к конкретным коммитам и запускам, номера которых указаны рядом с ними.

2. Ход работы

2.1. Шаги задания

Ниже — шаги из задания и то, как каждый из них выполнен в этом репозитории.

№	Шаг задания	Как выполнено
1	Python, <code>pip</code> , <code>virtualenv</code>	Python 3.11; окружение создаётся командой <code>python3 -m venv .venv</code>
2	Установить <code>virtualenv</code> , если нет	использован встроенный модуль <code>venv</code> , которого достаточно для задачи
3	Каталог проекта и окружение	каталог <code>vr-gaze-site/</code> , окружение <code>.venv/</code>
4	Зафиксировать зависимости, <code>.gitignore</code>	<code>requirements.txt</code> с точными версиями; <code>.gitignore</code> исключает окружение, <code>site/</code> , <code>_build/</code> , кэш
5	Установить MkDocs, создать каркас	MkDocs 1.6.1 с темой Material 9.7.6; настройки в <code>mkdocs.yml</code>
6	Локальная сборка	<code>mkdocs serve</code> для предпросмотра, затем <code>mkdocs build --strict</code>
7	Репозиторий и отправка на GitHub	<code>git init</code> , репозиторий <code>vr-gaze-site</code> на GitHub
8	GitHub Actions, Pages → Source = GitHub Actions	<code>.github/workflows/deploy.yml</code> , публикация через <code>upload-pages-artifact</code> и <code>deploy-pages</code>
9	Сервер или аккаунт на хостинге	аккаунт Cloudflare; токен API и идентификатор аккаунта хранятся в секретах репозитория (<code>CLOUDFLARE_API_TOKEN</code> , <code>CLOUDFLARE_ACCOUNT_ID</code>)
10	Скорректировать YAML под площадку	отдельное задание <code>deploy-cloudflare</code> : сборка с другим <code>SITE_URL</code> , выкладка командой <code>wrangler pages deploy</code>
11	Базовый URL	<code>site_url</code> берётся из переменной окружения <code>SITE_URL</code> ; <code>use_directory_urls: true</code> ; ссылки относительные
12	Проверка не только на глаз	шаг <code>healthcheck</code> в CI: код ответа HTTP 200 и контрольная строка в HTML; поиск и отображение без внешних CDN
13	Лицензии	контент — CC BY 4.0 (<code>LICENSE-CONTENT.md</code>), код — MIT (<code>LICENSE</code>)
14	Отладка	раздел Отладка
15	Собственный GitHub Action	репозиторий rsync-ssh-deploy : composite-action, теги <code>v1.0.0</code> и <code>v1</code> , лицензия MIT
16	Использование action в реальном репозитории	задание <code>deploy-ssh</code> в <code>deploy.yml</code> этого репозитория вызывает <code>777werona-afk/rsync-ssh-deploy@v1</code>

2.2. Базовый URL и подкаталог

MkDocs строит страницы с относительными ссылками, поэтому один и тот же `site/` работает и в корне домена, и в подкаталоге. Абсолютный адрес нужен только для `sitemap.xml` и канонических ссылок. Он задаётся переменной окружения:

```
1 site_url: !ENV [SITE_URL, "https://777werona-afk.github.io/vr-gaze-site/"]
```

В CI сайт собирается дважды: с адресом GitHub Pages и с адресом на Cloudflare Pages. Типичная ошибка при размещении в подкаталоге — абсолютные пути вида `/assets/...`; здесь их нет.

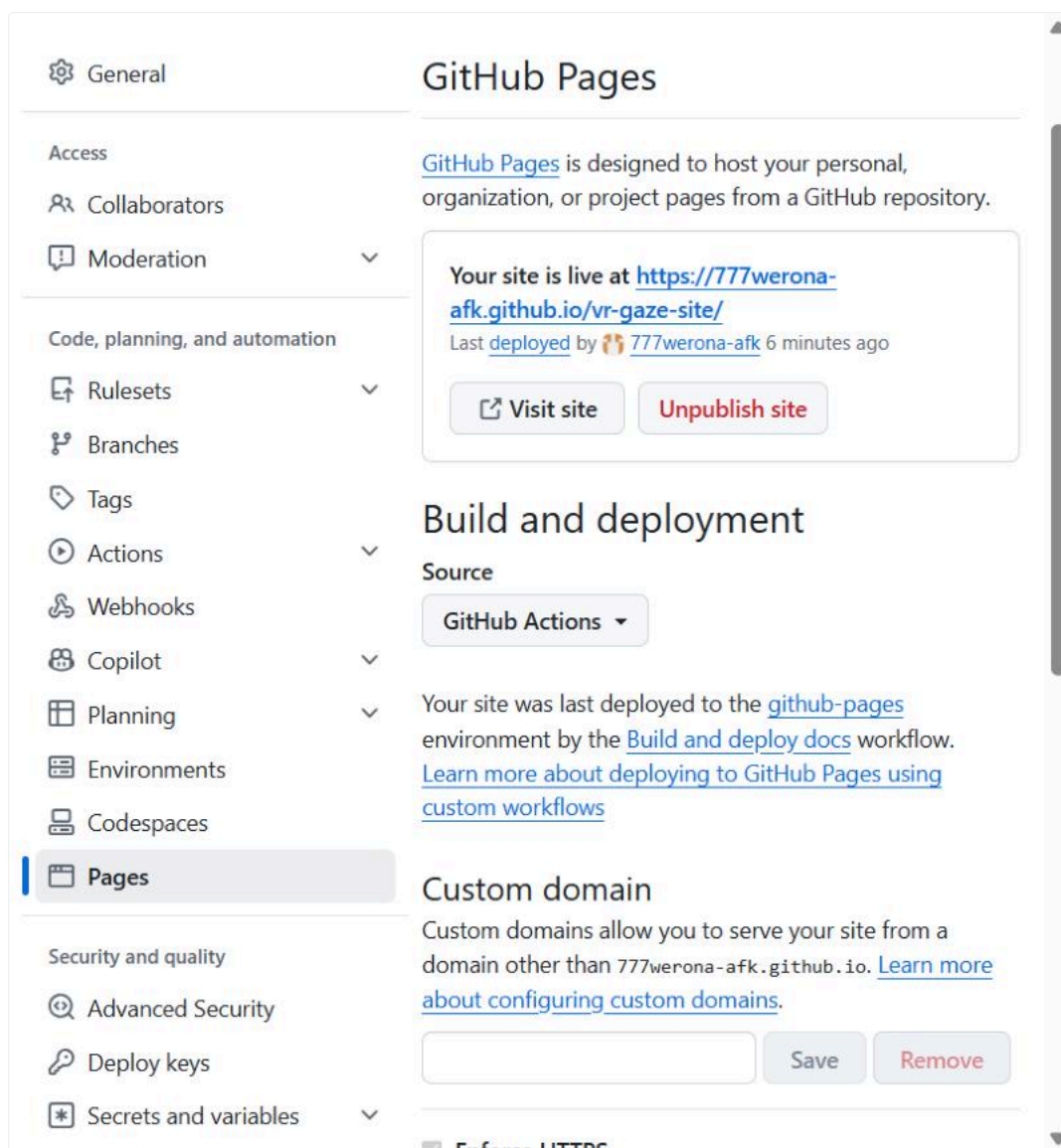
2.3. Проверка результата развёртывания

После выкладки пайплайн запрашивает опубликованный адрес и проверяет код ответа и наличие контрольной строки в HTML:

```
1 code=$(curl -s -o page.html -w '%{http_code}' "$URL")
2 test "$code" = "200"
3 grep -q 'Стабилизация взгляда в VR' page.html
```

Сайт не обращается к внешним серверам: шрифты отключены настройкой `font: false`, иконки и скрипты темы отдаются из самого сайта, формулы не используются. Поиск работает на клиенте с русской морфологией (`lang: ru`).

Настройки GitHub Pages: источник публикации — GitHub Actions, сайт доступен по адресу `https://777werona-afk.github.io/vr-gaze-site/`.



Успешный запуск с проверкой после выкладки (шаг `Healthcheck`) на обоих хостингах показан на скриншоте зелёного запуска в разделе «Отладка», полный запуск с тремя вариантами выкладки — в разделе 5.

2.4. Описание пайплайна

Полный текст `deploy.yml` с комментариями приведён в приложении А. Ниже — что делает каждое задание и зачем оно нужно.

Задание	Когда выполняется	Что делает	Почему так
<code>lint</code>	на каждый push и pull request	ставит зависимости (кэш <code>pip</code>), запускает <code>scripts/analyze.py</code> , собирает сайт в режиме <code>--strict</code> во временный каталог	ловит битые ссылки и ошибки расчёта до публикации; при ошибке остальные задания не запускаются
<code>build</code>	после <code>lint</code>	повторяет расчёт, собирает сайт дважды (адрес GitHub Pages и адрес Cloudflare), сохраняет два артефакта	два разных <code>SITE_URL</code> нужны для корректного <code>sitemap.xml</code> на каждом хостинге
<code>deploy-pages</code>	только <code>main</code> , не pull request	публикует артефакт через <code>actions/deploy-pages</code> , затем проверяет страницу по HTTP	права <code>pages: write</code> и <code>id-token: write</code> выданы только этому заданию
<code>deploy-cloudflare</code>	только <code>main</code> , не pull request	создаёт проект, выкладывает каталог командой <code>wrangler pages deploy</code> , проверяет страницу; без секретов шаги пропускаются	выкладка на shell-командах переносится между платформами без переписывания
<code>deploy-ssh</code>	только <code>main</code> , не pull request	выкладывает сайт собственным action <code>rsync-ssh-deploy@v1</code> на <code>sshd</code> , поднятый на раннере, и проверяет копию	показывает использование action в реальном workflow; замена <code>host</code> , <code>user</code> , <code>path</code> и ключа превращает его в выкладку на настоящий сервер

Принятые решения: права токена `contents: read` по умолчанию, расширяются только в заданиях выкладки; `concurrency` отменяет устаревший запуск для той же ветки; секреты не печатаются в лог, ключ SSH живёт во временном каталоге раннера и удаляется последним шагом action.

3. Исследовательская задача Т3. Обзор отечественных CI/CD-сервисов и статических хостингов

3.1. Постановка

Сравнить не менее трёх платформ CI/CD (GitVerse, SourceCraft, GitFlic) и не менее трёх вариантов размещения статического сайта (Helios ИТМО, Yandex Object Storage, Timeweb Cloud / Selectel). Отдельно оценить трудозатраты на перенос уже написанного workflow GitHub Actions.

3.2. Метод и ограничения

Сведения взяты из официальной документации платформ (ссылки в конце раздела) на момент подготовки отчёта. Раздел отмечает, что именно подтверждено документацией, а что не удалось проверить: в таблицах такие ячейки помечены словами «не проверено». Платформы я не тестировала запуском одного и того же пайплайна, поэтому сравнение документальное, а оценка миграции — экспертная.

3.3. Платформы CI/CD

Критерий	GitVerse	SourceCraft	GitFlic
Файл пайплайна	<code>.gitverse/workflows/*.yaml</code>	<code>.sourcecraft/ci.yaml</code>	<code>.gitflic-ci.yaml</code>
Модель описания	jobs / steps, <code>runs-on</code> , <code>uses</code> , <code>run</code>	events (<code>on</code>) → workflows → tasks → cubes	stages / jobs
Совместимость	документация заявляет совместимость с синтаксисом GitHub Actions, <code>actions/checkout@v4</code> указан в примере	собственный синтаксис; есть интерпретация с GitHub Actions и совместимость с GitLab-пайплайнами (по документации)	собственный синтаксис, файл назван не как у GitLab
Раннеры	облачные, self-hosted (на репозиторий), организационные; выбор по меткам <code>runs-on</code>	воркеры (workers) платформы; есть self-hosted worker	self-hosted агенты (Shell, PowerShell, Docker) и облачные агенты в Docker или Kubernetes
Секреты	секреты и переменные на уровне репозитория и организации	отдельная система секретов; переменные на уровнях global / workflow / task / cube	Vault для секретов, переменные окружения
Артефакты и кэш	артефакты и кэш поддерживаются; лимиты ниже	не проверено	артефакты и кэш документированы отдельными разделами
Каталог готовых действий	стартовый репозиторий workflow-примеров; действия в стиле GitHub (<code>uses:</code>)	кубы и подключение GitHub Actions	шаблоны конфигураций и примеры
Лимиты бесплатного тарифа	500 мин. (приватные) и 1000 мин. (публичные) на пользователя или организацию; для верифицированных организаций вдвое больше; задача на облачном раннере — до 30 мин.; артефакты — 500 МБ на все репозитории, хранение 30 дней	не проверено	не проверено
Собственный домен и HTTPS	не относится к CI/CD (зависит от хостинга)	не относится к CI/CD	не относится к CI/CD

⚠ Важное изменение в GitVerse

Документация GitVerse предупреждает, что адрес <https://gitverse.ru/sc> перестанет поддерживать локальные раннеры в сентябре 2026 года; раннеры, зарегистрированные по старому адресу, нужно перерегистрировать на <https://gitverse.ru>.

3.4. Варианты размещения

Критерий	Helios ИТМО	Yandex Object Storage (хостинг сайта)	Timeweb Cloud S3	Selectel (облачное хранилище S3)
Способ доставки	SSH / rsync (по формулировке задания Р4)	S3 API, совместимые клиенты	S3 API, CLI, Cyberduck, браузер	S3 API и Swift API, FTP, rclone, AWS CLI, s3cmd
Адрес сайта	подкаталог на сервере университета	<code>https://<бакет>.website.yandexcloud.net</code>	через публичный доступ к бакету	не проверено
HTTPS	не проверено	включён автоматически на <code>website.yandexcloud.net</code> , HTTP перенаправляется на HTTPS	доступ по HTTP и HTTPS	не проверено
Собственный домен	не проверено	возможен, но имя домена должно совпадать с именем бакета	поддерживается привязка домена	не проверено
Особенности	сайт живёт в подкаталоге: нужны корректные относительные ссылки	бакет должен быть публичным, иначе ответ 403; TLS 1.0 и 1.1 отключены с 1 августа 2025	документация прямо называет хранилище вариантом для статических сайтов	в изученном разделе документации хостинг сайтов не описан
Стоимость	для студентов — в рамках ресурсов университета (уточнять)	оплата по тарифам хранения и трафика	оплата по тарифам	оплата по тарифам

Про Helios публичной документации найти не удалось: условия доступа, HTTPS и поддерживаемые способы загрузки нужно уточнять в материалах курса и у преподавателя.

3.5. Оценка миграции workflow с GitHub Actions

Мой workflow выполняет четыре действия: сборка сайта, публикация на GitHub Pages, выкладка на Cloudflare Pages командой `wrangler pages deploy` и проверка опубликованной страницы. Оценка для переноса на GitVerse как на платформу, декларирующую совместимость:

Часть workflow	Что с ней происходит
Структура <code>on / jobs / steps</code> , <code>runs-on</code> , <code>run</code>	переносится практически без изменений; файл переезжает из <code>.github/workflows/</code> в <code>.gitverse/workflows/</code>
<code>actions/checkout</code> , <code>actions/setup-python</code>	первое подтверждено документацией, второе нужно проверить; при отсутствии заменяется шагом <code>run: pip install ...</code> на образе с Python
<code>actions/cache</code>	нужно проверить; кэш платформы описан в её документации и может иметь другой синтаксис
Секреты <code>\${{ secrets.NAME }}</code>	синтаксис тот же, но секреты заводятся заново в настройках платформы
Контекст <code>github.*</code> и <code>GITHUB_TOKEN</code>	заменяется на контекст <code>GitVerse</code> ; логику, завязанную на токен <code>GitHub</code> , придётся переписать
<code>upload-pages-artifact</code> + <code>deploy-pages</code> , <code>permissions: pages/id-token</code>	принципиально не переносится: это публикация через API <code>GitHub Pages</code> и <code>OIDC</code> -токен <code>GitHub</code>
<code>environment: github-pages</code>	не переносится
Выкладка <code>wrangler pages deploy</code> и проверка <code>curl</code>	переносятся без изменений, потому что это обычные <code>shell</code> -команды; меняется только способ хранения токена

Итог: структура пайплайна и `shell`-часть переезжают почти даром, а публикацию на `GitHub Pages` нужно заменить на доставку по `SSH` или `S3`. Для `SourceCraft` и `GitFlic` работа больше: оба используют собственную структуру описания, и пайплайн придётся переписывать целиком, перенося только содержимое `run`-шагов.

3.6. Вывод по ТЗ

Для проекта, где пайплайн — это «установить зависимости, собрать сайт, выложить файлы», наименьшие затраты на перенос у `GitVerse` из-за заявленной совместимости с `GitHub Actions`. Доставку лучше строить на `shell`-командах (`wrangler`, `rsync`, `curl`), а не на специфичных для платформы действиях: тогда смена платформы затрагивает только обвязку.

3.7. Источники

- [GitVerse: CI/CD, справочник, быстрый старт, раннеры, лимиты](#)
- [SourceCraft: CI/CD, пример с GitHub Actions в Yandex Cloud](#)
- [GitFlic: документация, справочник .gitflic-ci.yaml](#)
- [Yandex Object Storage: хостинг статического сайта](#)
- [Timeweb Cloud: S3-хранилище](#)
- [Selectel: облачное хранилище](#)

4. Практическая задача РЗ. Воспроизводимый конвейер «данные → результат → сайт»

4.1. Задача

Подготовить скрипт, который читает исходные данные эксперимента, выполняет расчёт и формирует графики и таблицы, и встроить его вызов в сборку сайта. Показать три вещи: обновление сайта при изменении данных, кэширование без лишнего пересчёта и метку версии на странице.

4.2. Данные

Входные данные — 8 CSV-логов взгляда из VR-музея, записанных 16 сентября 2026 года моей пилотной системой одной кнопкой (`gaze_session_20260916_*.csv`). Формат: `timestamp`, `gaze_x`, `gaze_y`, `gaze_z` (вектор направления взгляда), `pupil` (диаметр зрачка), `openness` (раскрытие глаза). Всего 17 132 строки, 206 секунд записи. Файлы лежат в `data/raw/`.

Что показал первый просмотр данных:

- Частота записи в семи сессиях около 72 Гц, в самой первой — около 370 Гц.
- Первая сессия (8 с) полностью статична: вектор взгляда не меняется ни в одной строке. Скрипт помечает такие сессии как «статичные» и не считает для них эффект стабилизации.
- Столбцы `pupil` и `openness` во всех сессиях равны 1,0 и информации не несут, поэтому в расчёте не используются.
- В пяти сессиях из восьми есть мгновенные разрывы направления взгляда, всего 32: между соседними кадрами вектор поворачивается на 42–174°, большинство — на 125–175°. Для глаз и головы это физически невозможно (при 72 Гц это тысячи градусов в секунду), поэтому я считаю такие шаги разрывом сигнала.

Скрипт терпим к особенностям загрузки: сам определяет разделитель, регистр названий колонок, BOM в начале файла и единицы времени (секунды, миллисекунды, микросекунды, наносекунды).

4.3. Схема конвейера



В Makefile расчёт — зависимость сборки: `make strict` сначала вызывает `scripts/analyze.py`, затем `mkdocs build --strict`. В CI те же две команды выполняются на чистом раннере.

4.4. Что считает скрипт

Направление взгляда переводится в два угла (горизонтальный и вертикальный, градусы). Шаги больше 30° между соседними кадрами вырезаются, а оставшиеся сегменты склеиваются: иначе ступенька попадает в спектр и ложно увеличивает «тремор». Затем сигнал приводится к равномерной сетке по медианному шагу времени и считаются:

- **тремор** — RMS сигнала в полосе 4–12 Гц (полосовой фильтр Баттерворта 2-го порядка) по обоим углам, градусы;
- **джиттер** — RMS всего, что быстрее 2 Гц; сюда входят и быстрые произвольные движения головы, поэтому как оценку тремора я использую его осторожно;
- **частота пика** спектра в полосе тремора (метод Уэлча), только если в полосе есть настоящий локальный максимум;
- **медианная угловая скорость** взгляда;

- те же метрики после стабилизатора — фильтра One Euro (`mincutoff=1.0`, `beta=0.001`, `dcutoff=1.0`). Это простой эталонный фильтр для сравнения, а не моя обученная модель-стабилизатор.

4.5. Результаты

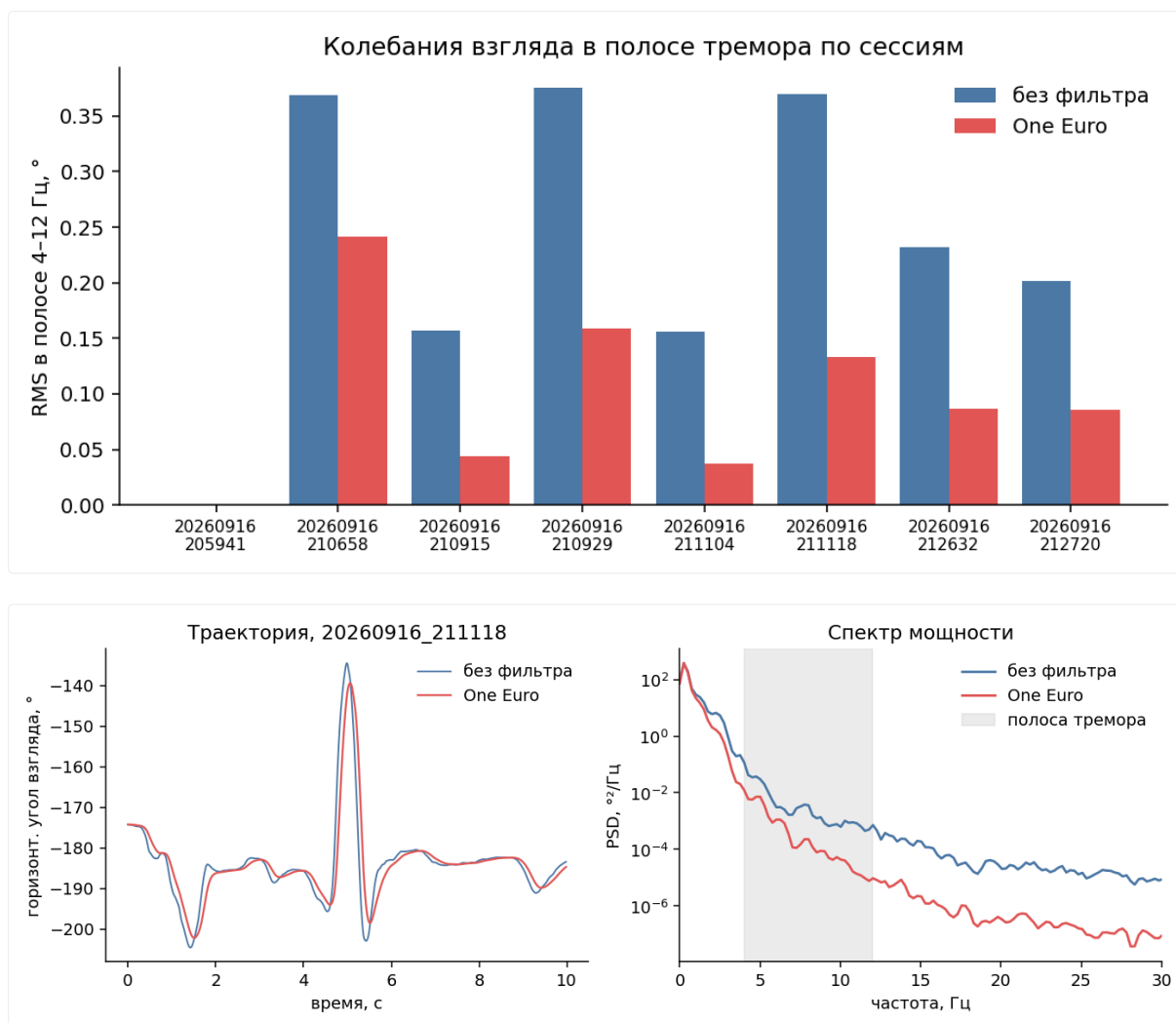
Параметр	Значение
Коммит	fc452ee
Дата сборки (UTC)	2026-10-05 12:20:56
Версия набора данных	pilot-2026-09-16
SHA-256 набора данных	4a161632da0e2ed9...
Файлов / строк	8 / 17132
Версия скрипта	1.0

4.5.1. Сводная таблица по сессиям

Сессия	Строк	Длит., с	Частота, Гц	Разрывов	Скорость, °/с	Тремор до, °	Тремор после, °
20260916_205941 (статичная)	2938	8,0	370,4	0	0,0	0,000	0,000
20260916_210658	2866	39,9	71,9	11	19,3	0,369	0,242
20260916_210915	471	6,5	71,9	0	4,3	0,157	0,044
20260916_210929	2575	35,9	71,9	11	23,3	0,375	0,159
20260916_211104	326	4,8	71,9	0	8,5	0,156	0,038
20260916_211118	2898	40,4	71,9	6	20,2	0,370	0,133
20260916_212632	2678	37,4	71,9	3	28,7	0,232	0,087
20260916_212720	2380	33,1	71,9	1	36,5	0,202	0,085
Итого / среднее по 7 сессиям с движением	17132	206,0	71,9	32	20,1	0,266	0,113

«Тремор» — RMS в полосе 4–12 Гц, «Снижение» показывает, насколько фильтр уменьшил этот показатель. Средние значения посчитаны только по сессиям, где взгляд двигался.

4.5.2. Графики



4.5.3. Чувствительность к параметру фильтра

Параметр `beta` определяет, как быстро фильтр ослабляет сглаживание при быстром движении. Я проверила три значения на всех сессиях с движением:

<code>beta</code>	Среднее снижение тремора, %	Сессий со снижением	Средняя задержка, мс
0	78,7	7 из 7	123
0,001	60,6	7 из 7	91
0,007	13,4	4 из 7	52

При `beta=0` фильтр — обычный низкочастотный с частотой среза 1 Гц: тремор подавляется сильнее всего, но взгляд отстаёт примерно на 120 мс, и для навигации в VR это много. При `beta=0,007` задержка вдвое меньше, но при быстрых поворотах головы фильтр почти перестаёт сглаживать: в трёх сессиях из семи тремор в полосе 4–12 Гц даже вырос. Значение `0,001` я оставила как компромисс между подавлением тремора и задержкой. Подбор параметров сделан на тех же данных, на которых измерен результат, поэтому цифры — оценка возможностей простого фильтра, а не проверка на новых данных.

4.6. Как воспроизвести

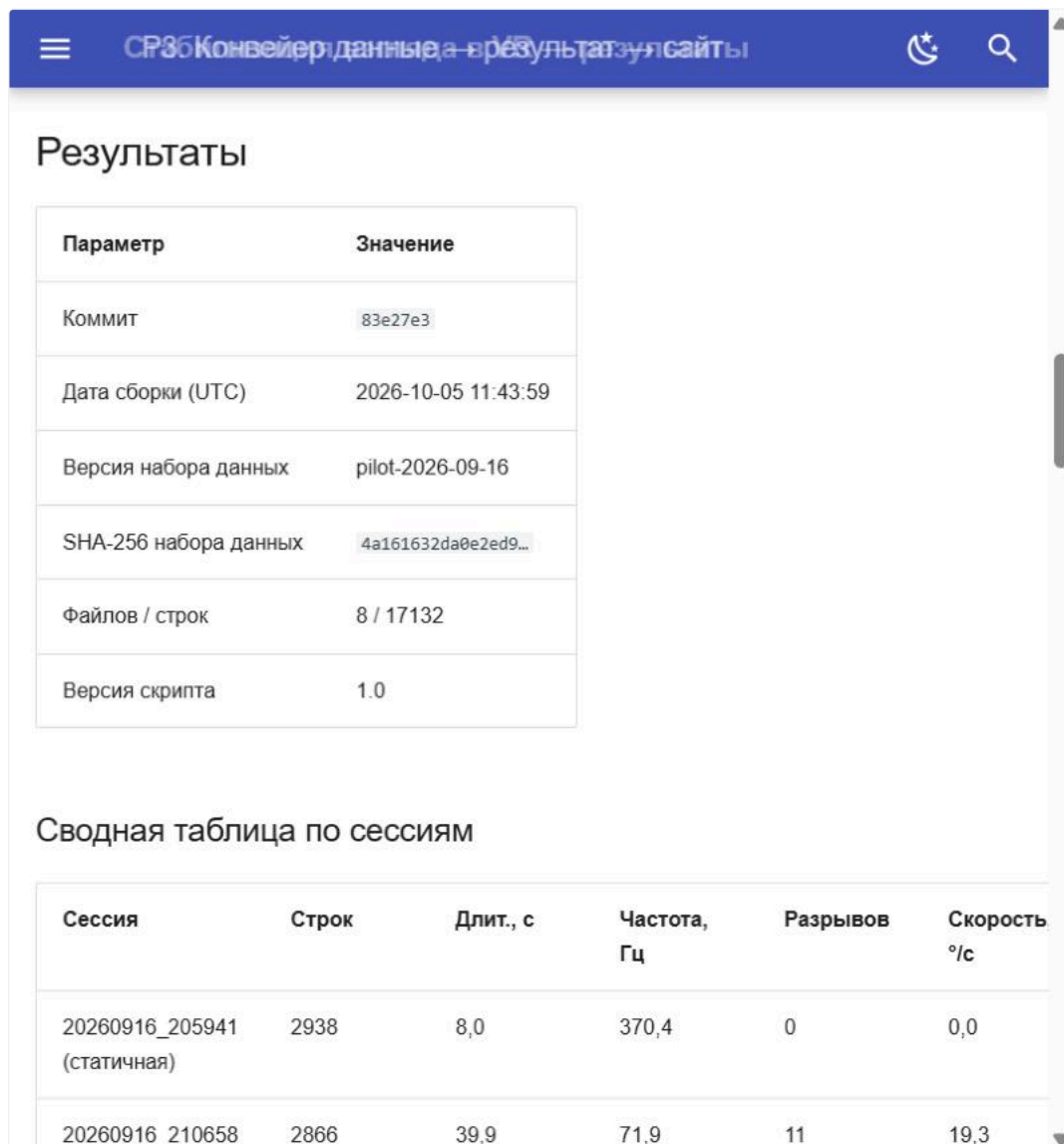
```
1 python3 -m venv .venv && source .venv/bin/activate
2 pip install -r requirements.txt
3 make strict      # расчёт + сборка сайта в строгом режиме
4 make serve       # локальный предпросмотр
```

4.7. Демонстрация требований РЗ

4.7.1. Изменение данных → push → обновление сайта

В CI на каждый push в `main` запускается `scripts/analyze.py`, поэтому новая версия CSV в `data/raw/` меняет таблицу, графики и метку версии без ручных действий.

Проверила на реальном запуске. Перед изменением сайт был собран из коммита `83e27e3`: 8 файлов, 17 132 строки, SHA-256 набора данных `4a161632da0e2ed9...`.



Результаты

Параметр	Значение
Коммит	83e27e3
Дата сборки (UTC)	2026-10-05 11:43:59
Версия набора данных	pilot-2026-09-16
SHA-256 набора данных	4a161632da0e2ed9...
Файлов / строк	8 / 17132
Версия скрипта	1.0

Сводная таблица по сессиям

Сессия	Строк	Длит., с	Частота, Гц	Разрывов	Скорость %с
20260916_205941 (статичная)	2938	8,0	370,4	0	0,0
20260916_210658	2866	39,9	71,9	11	19,3

Затем я удалила из `data/raw/` одну сессию (`gaze_session_20260916_210658.csv`, 2 866 строк, 11 разрывов) и запустила коммит `ade4237` в `main`. Запуск №7 прошёл за 1 мин 40 с, сайт обновился без ручных действий: в метке версии стали

коммит `ade4237` , 7 файлов, 14 266 строк (17 132 – 2 866) и другой SHA-256 набора данных `c7d0a833f36cbf8e...` . Из сводной таблицы пропала строка этой сессии.

РЗ. Конвейер данные → результат → сайт

Результаты

Параметр	Значение
Коммит	<code>ade4237</code>
Дата сборки (UTC)	2026-10-05 11:50:25
Версия набора данных	<code>pilot-2026-09-16</code>
SHA-256 набора данных	<code>c7d0a833f36cbf8e...</code>
Файлов / строк	7 / 14266
Версия скрипта	1.0

Сводная таблица по сессиям

Сессия	Строк	Длит., с	Частота, Гц	Разрывов	Скорость %с
20260916_205941 (статичная)	2938	8,0	370,4	0	0,0
20260916_210915	471	6,5	71,9	0	4,3

После проверки я вернула файл коммитом-откатом (`git revert ade4237`), поэтому в текущей версии сайта снова 8 сессий и 17 132 строки.

4.7.2. Кэширование и замер выигрыша

Скрипт считает ключ кэша из хеша SHA-256 входных CSV, хеша самого скрипта и параметров обработки. Если ключ совпал с сохранённым в `results/.cache.json`, тяжёлый расчёт пропускается, обновляются только коммит и дата в метке версии.

Замер локальный (`make bench`): три запуска с `--force` и три повторных без изменений на моих 8 сессиях:

Режим	Время расчёта, с	Что происходит
Холодный (<code>--force</code>)	0,72 / 0,75 / 0,91	чтение CSV, расчёт метрик, три прогона фильтра, графики
Тёплый (кэш)	0,01 / 0,01 / 0,01	совпал ключ — пересчёт пропущен

Замер сделан в облачной среде подготовки, а не на моём компьютере и не на раннере CI. На 17 тысячах строк выигрыш небольшой в абсолютных секундах; кэш станет заметным на датасете из 50 участников, где расчёт займёт минуты.

Замер в CI на GitHub Actions (job `lint`, раннер `ubuntu-latest`, Python 3.11) для двух первых запусков:

Запуск	Кэш pip	Install dependencies	Расчёт	Весь job lint
№1, коммит <code>46be2a4</code>	нет (первый запуск, кэш сохраняется)	27 с	4 с	41 с
№2, коммит <code>7977203</code>	попадание (109 МБ восстановлено)	19 с	2 с	28 с

Кэш зависимостей (`cache: pip` в `actions/setup-python`) сократил установку на 8 секунд, а весь job на 13 секунд (около 32%). Расчёт в CI занимает 2–4 секунды при обоих запусках: кэш результатов лежит в `results/.cache.json`, этот файл в репозиторий не коммитится, поэтому на чистом раннере расчёт каждый раз выполняется заново. Разница между 4 и 2 секундами — разброс раннера, а не эффект кэша. Для двух запусков это одиночные замеры, статистики из них не получится.

4.7.3. Метка версии на странице

Блок «Метка версии» выше формируется скриптом и содержит хеш коммита, дату сборки в UTC, версию набора данных (из файла `data/DATASET_VERSION`) и SHA-256 всех входных CSV. По нему опубликованный результат однозначно связывается с версией кода и данных.

4.8. Ограничения

- Таблица и графики описывают одну пилотную запись одного автора (8 сессий, 206 секунд). Сравнение групп с ОВЗ и без ОВЗ — задача будущего сбора данных, пилотных данных для него недостаточно.
- Метки «тремор» и «джиттер» — оценки по сигналу взгляда в градусах и по спектру; в данных остаются быстрые произвольные движения головы, поэтому для клинических выводов эти значения не предназначены.
- Порог разрыва 30° выбран по виду данных: обычные шаги между кадрами не превышают 7°, а следующие по величине — 42° и больше, так что значения между 7° и 42° в записи не встречаются. Причину разрывов я не устанавливала.
- Стабилизатор в расчёте — эталонный фильтр One Euro, а не обученная модель; сравнение с моделью — отдельная работа.

5. GitHub Action rsync-ssh-deploy

5.1. Постановка

Нужно оформить action для выкладки каталога на сервер по SSH. Требования и то, где каждое из них выполнено:

Требование	Как выполнено	Где смотреть
composite или Docker-action с корректным <code>action.yml</code> (inputs, outputs, описание, branding)	<code>composite-action</code> ; <code>inputs</code> , <code>outputs</code> , <code>description</code> , <code>author</code> , <code>branding</code> (иконка <code>upload-cloud</code> , цвет <code>blue</code>)	action.yml , приложение А
параметры: хост, путь назначения, исходный каталог, ключ, флаг <code>--delete</code>	<code>host</code> , <code>path</code> , <code>source</code> , <code>key</code> , <code>delete</code> ; дополнительно <code>user</code> , <code>port</code> , <code>known_hosts</code> , <code>dry_run</code>	таблица ниже
README с примером и совместимостью	в README: пример workflow, таблицы параметров и результатов, раздел «Совместимость»	README
семантические теги <code>v1.0.0</code> и подвижный <code>v1</code>	релиз <code>v1.0.0</code> ; workflow <code>Move major tag</code> переставляет <code>v1</code> на коммит релиза	теги
лицензия	MIT	LICENSE
публикация в Marketplace	не выполнено: публикация требует принять соглашение разработчика Marketplace владельцем аккаунта; action полностью работоспособен по ссылке <code>777werona-afk/rsync-ssh-deploy@v1</code>	см. раздел 5.6
ссылка на реальный репозиторий, где action используется	репозиторий этого отчёта, задание <code>deploy-ssh</code>	vr-gaze-site

5.2. Устройство action

Action — composite: четыре шага `bash` внутри одного `action.yml`, Docker не нужен, поэтому запуск не тратит время на сборку образа.

Шаг	Что делает
Check inputs	проверяет, что параметры заданы, исходный каталог существует и не пуст; в режиме <code>delete</code> отклоняет опасные пути (<code>/</code> , <code>.</code> , <code>~</code>)
Prepare SSH key	записывает ключ во временный каталог раннера с правами <code>600</code> , готовит <code>known_hosts</code> (из параметра или через <code>ssh-keyscan</code> с предупреждением)
Sync with rsync	выполняет <code>rsync -az --stats</code> (с <code>--delete</code> , если включён флаг), читает статистику и записывает <code>outputs</code>
Remove key	выполняется всегда (<code>if: always()</code>) и удаляет ключ даже после ошибки

Параметры:

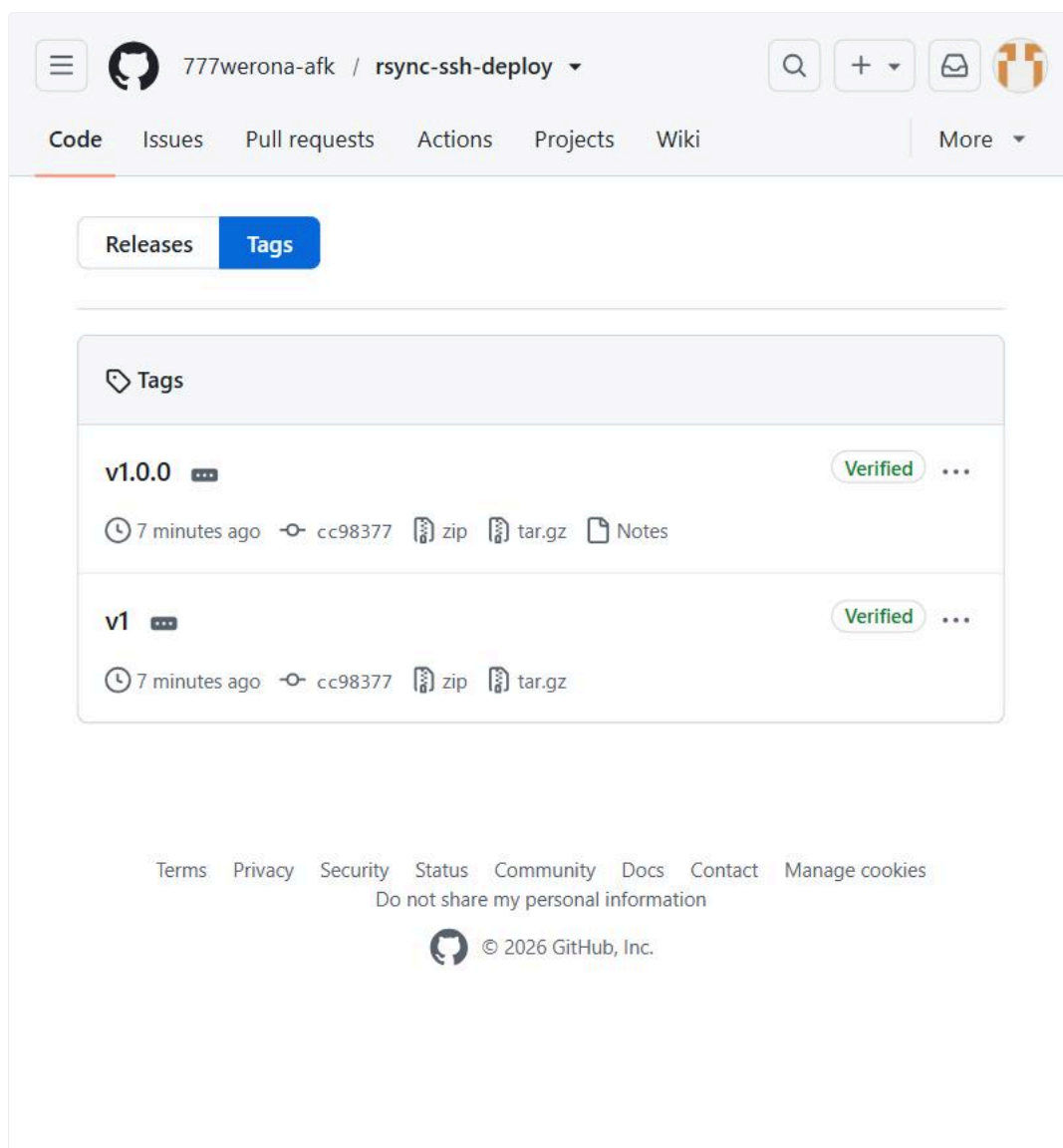
Параметр	Обязательный	По умолчанию	Назначение
host	да		сервер
user	да		пользователь на сервере
path	да		каталог назначения
source	нет	site	исходный каталог
key	да		закрытый SSH-ключ
delete	нет	false	<code>rsync --delete</code> : удалить на сервере лишние файлы
port	нет	22	порт SSH
known_hosts	нет	пусто	строки <code>known_hosts</code> для проверки сервера
dry_run	нет	false	только показать план

Результаты (outputs): `destination` (формат `user@host:path`), `files_transferred`, `files_deleted`.

Защита от типичных ошибок выложена в README: пустой источник вместе с `delete` стёр бы сайт на сервере, поэтому он останавливает выкладку; проверка сервера идёт с `StrictHostKeyChecking=yes`.

5.3. Версионирование

Релиз `v1.0.0` создан на коммите `cc98377`. После публикации тега сработал workflow `Move major tag` и поставил подвижный тег `v1` на тот же коммит. Теперь `uses: 777werona-afk/rsync-ssh-deploy@v1` получает все совместимые исправления `v1.x.y`, а `@v1.0.0` фиксирует точную версию.



```
1 on:
2   push:
3     tags:
4       - 'v[0-9]+.[0-9]+.[0-9]+'
5   # внутри задания:
6   #   major="${GITHUB_REF_NAME%.*}"; git tag -f "$major" "$GITHUB_SHA"; git push -f origin "$major"
```

5.4. Проверка action

Workflow `Test action` на каждый push поднимает на раннере `sshd`, создаёт ключ и выполняет три проверки:

Проверка	Ожидаемый результат	Итог
первая выкладка каталога с подкаталогом	файлы появились на сервере	пройдена
вторая выкладка после удаления файла из источника и появления лишнего файла на сервере, <code>delete: 'true'</code>	лишние файлы удалены, <code>destination</code> совпадает с ожидаемым	пройдена
выкладка из пустого каталога	action завершается ошибкой	пройдена (сообщение «Исходный каталог 'empty' пуст...»)

Запуск №1 завершился успехом за 18 секунд (коммит `cc98377`). Две «ошибки» в аннотациях — это ожидаемый отказ на пустом источнике (шаг с `continue-on-error`).

← Test action

Create test.yml #1

Summary ▾

Triggered via push 1 minute ago

777werona-afk pushed cc98377 main

Status	Total duration	Artifacts
Success	<u>18s</u>	—

test.yml
on: push

test 15s

Annotations
2 errors, 3 warnings, and 1 notice

test
Process completed with exit code 1.

5.5. Использование в реальном репозитории

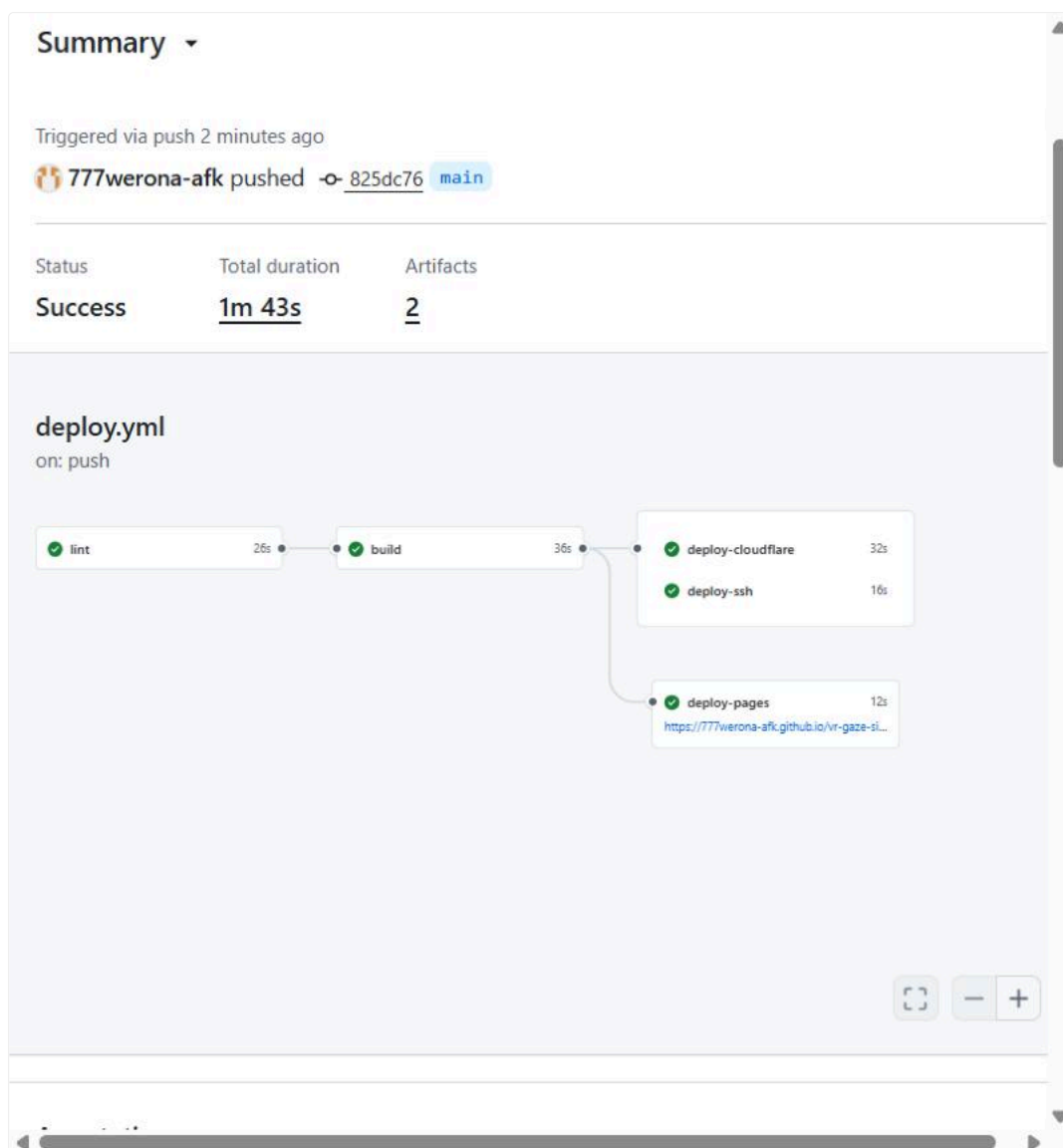
В `deploy.yml` репозитория `vr-gaze-site` добавлено задание `deploy-ssh`, которое скачивает собранный сайт и выкладывает его action'ом:

```
1 - name: Deploy over SSH with rsync-ssh-deploy
2   id: deploy
3   uses: 777werona-afk/rsync-ssh-deploy@v1
4   with:
5     host: localhost
6     user: ${ steps.ssh.outputs.user }
7     path: /tmp/site-ssh
8     source: site-hosting
9     key: ${ steps.ssh.outputs.key }
10    delete: 'true'
```

Что это доказывает и чего не доказывает

У проекта нет постоянного SSH-сервера, поэтому `host: localhost`: сайт выкладывается на `sshd`, поднятый на самом раннере, и проверяется там же (файл `index.html` существует и содержит контрольную строку). Так проверены работа action в чужом репозитории по тегу `@v1` и обмен параметрами и результатами. Выкладка на постоянный удалённый сервер не проверялась. Чтобы выкладывать на настоящий сервер, достаточно заменить `host`, `user`, `path` и ключ на секреты репозитория.

Запуск №9 (коммит `825dc76`) прошёл полностью: все пять заданий зелёные, `deploy-ssh` занял 16 секунд.



5.6. Публикация в Marketplace

На момент подготовки отчёта action **не опубликован в GitHub Marketplace**. Всё необходимое для публикации в репозитории есть: `action.yml` с `name`, `description`, `author` и `branding`, README, лицензия MIT и релиз `v1.0.0`. Публикация делается кнопкой «Publish this release to the GitHub Marketplace» на странице релиза, но сначала владелец аккаунта должен принять соглашение для разработчиков Marketplace и включить двухфакторную аутентификацию. Пока этого не сделано, action подключается напрямую по имени репозитория и тегу: `uses: 777werona-afk/rsync-ssh-deploy@v1`. Это работает для любого публичного репозитория и подтверждено запуском №9.

5.7. Ограничения

- Windows-паннеры не поддерживаются: нужны `bash` и `rsync`.
- Без параметра `known_hosts` отпечаток сервера принимается при первом подключении, это менее безопасно; в README рекомендуется передавать `known_hosts` через секрет.
- Action проверялся на `ubuntu-latest` и на локальном `ssh`; другие серверы и платформы CI (например, GitVerse) не проверялись.

6. Измерения

Все значения взяты из реальных запусков GitHub Actions (раннер `ubuntu-latest`, Python 3.11) и с опубликованных сайтов. Длительности заданий — как их показывает GitHub (округление до секунды).

6.1. Время заданий по запускам

Запуск	Коммит	lint, c	build, c	deploy- pages, c	deploy- cloudflare, c	deploy- ssh, c	Весь запуск
№4 (повторный, после добавления секретов)	93ca770	27	25	9	31	—	1 мин 34 с
№5 (намеренная ошибка)	845bd78	36	пропущено	пропущено	пропущено	—	39 с
№6	83e27e3	32	38	11	33	—	1 мин 52 с
№7 (данные изменены)	ade4237	22	38	9	32	—	1 мин 40 с
№8 (откат изменения данных)	22f54bb	30	36	11	32	—	1 мин 45 с
№9 (добавлено задание deploy-ssh)	825dc76	26	36	12	32	16	1 мин 43 с
№10 (итоговая версия отчёта)	152208f	31	37	29	36	15	1 мин 49 с
Среднее по успешным запускам		28,0	35,0	13,5	32,7	15,5 (два запуска)	1 мин 44 с (103,8 с)
Минимум / максимум		22 / 32	25 / 38	9 / 29	31 / 36	15 / 16	94 / 112 с

Запуск №5 остановился на `lint`: ошибка в строгом режиме за 36 секунд не дала собрать и выложить сайт. Время всего запуска меньше суммы заданий, потому что `deploy-pages`, `deploy-cloudflare` и `deploy-ssh` идут параллельно после `build`. Для шести успешных запусков это одиночные замеры: разброс 22–32 секунды у `lint` объясняется работой раннера, статистической оценки из них не получится. В запуске №10 `deploy-pages` занял 29 секунд вместо обычных 9–12; причину я не выясняла (шаг выкладки на стороне GitHub), поэтому среднее по этому заданию завышено одним выбросом.

6.2. Разбивка по шагам (запуск №9)

Задание	Шаг	Время, с
<code>build</code>	установка зависимостей (<code>pip install</code> , кэш <code>pip</code>)	20
<code>build</code>	расчёт (<code>scripts/analyze.py</code>)	3
<code>build</code>	сборка для GitHub Pages	1
<code>build</code>	сборка для Cloudflare Pages	0
<code>build</code>	<code>checkout</code> (1), <code>setup-python</code> (3), загрузка двух артефактов (3), служебные шаги (2)	9
<code>build</code>	накладные расходы между шагами (36 с задания – 33 с сумма шагов)	3
<code>deploy-pages</code>	<code>actions/deploy-pages</code>	7
<code>deploy-pages</code>	проверка страницы (<code>Healthcheck</code>)	1
<code>deploy-cloudflare</code>	<code>download-artifact</code>	1
<code>deploy-cloudflare</code>	создание проекта (<code>wrangler pages project create</code>)	10
<code>deploy-cloudflare</code>	выкладка (<code>wrangler pages deploy</code>)	8
<code>deploy-cloudflare</code>	проверка страницы (<code>Healthcheck</code> , внутри пауза 10 с)	10
<code>deploy-ssh</code>	запуск <code>sshd</code> и создание ключа	7
<code>deploy-ssh</code>	выкладка action'ом <code>rsync-ssh-deploy</code>	2
<code>deploy-ssh</code>	проверка копии	1

Самый дорогой шаг сборки — установка зависимостей (20 из 36 с). Шаг «создание проекта» на Cloudflare выполняется при каждом запуске и каждый раз завершается ошибкой «проект уже есть», которую подавляет `|| true`: после первой выкладки его можно убрать и сэкономить 10 секунд.

6.3. Время развёртывания по вариантам выкладки

Вариант	Полезная часть выкладки, с	Всё задание, с	Что входит в разницу
GitHub Pages	7	12	подготовка раннера, проверка страницы (1 с)
Cloudflare Pages	8	32	создание проекта (10 с), проверка страницы с паузой на публикацию (10 с), подготовка раннера
SSH (<code>rsync - ssh-deploy</code>)	2	16	запуск <code>sshd</code> и создание ключа (7 с), проверка копии, подготовка раннера; на реальном сервере <code>sshd</code> уже работает

Таким образом, сами операции выкладки занимают от 2 до 8 секунд; остальное — подготовка и проверка.

6.4. Вес страниц

Вес измерен на опубликованных сайтах (версия `152208f` , запуск №10) скриптом в браузере: для каждой страницы запрашиваются сама страница и все её ресурсы с того же сайта (стили, скрипты, значок, картинки), размеры берутся из `PerformanceResourceTiming` . «Передано» — байты по сети (со сжатием, которое применяет хостинг), «Развёрнуто» — после распаковки. Кэш браузера отключён, поэтому цифры соответствуют первому заходу на страницу. Внешних запросов нет.

Страница	Ресурсов, шт.	Только HTML, передано, Б	Всего, GitHub Pages, передано, Б	Всего, Cloudflare Pages, передано, Б	Всего, развёрнуто, Б
/	5	7 177	70 449	71 805	296 936
intro/	5	7 062	70 334	71 752	296 632
workflow/	6	8 927	157 531	158 986	388 717
research/t3-ci-cd/	5	9 612	72 884	74 349	306 223
practice/p3-pipeline/	9	12 200	318 142	319 512	560 415
practice/action/	8	10 044	186 942	188 203	423 435
measurements/	5	7 645	70 917	72 339	299 852
debug/	8	10 371	233 653	235 042	470 127
conclusion/	5	8 028	71 300	72 696	298 962
links/	5	6 472	69 744	71 156	294 833
listings/	5	19 888	83 160	81 477	410 677
Среднее по 11 страницам		9 766	127 732	128 847	367 892

Поисковый индекс `search/search_index.json` в таблицу не входит: он загружается отдельно, один раз при первом поиске: на GitHub Pages 38 667 Б по сети (219 529 Б развёрнуто), на Cloudflare Pages 46 264 Б по сети.

Что видно из таблицы:

- Основной вес страницы — не текст, а общие ресурсы темы (стили и скрипты, около 65 КБ по сети на любой странице). Без картинок страница весит 70–83 КБ.
- Самые тяжёлые страницы — те, где есть графики и скриншоты: `practice/p3-pipeline/` (около 318 КБ) и `debug/` (около 234 КБ). Листинги (`listings/`) тяжелы по тексту (HTML 140 КБ развёрнуто), но хорошо сжимаются.
- Размеры «Всего» на двух хостингах отличаются на 1–2 КБ у страниц и на несколько КБ у поискового индекса: хостинги сжимают ответы по-разному. Картинки и стили одинаковы.
- Для сравнения: на версии `825dc76` до добавления отчётных разделов главная страница весила 69 257 Б по сети, сейчас 70 449 Б; рост связан с титульным листом и таблицей разделов.

6.5. Другие измерения

Время расчёта с кэшем и без, а также время `lint` с кэшем `pip` и без него — в [разделе 4](#). Время тестового workflow action: 18 секунд (запуск №1 репозитория `rsync-ssh-deploy`, из них 15 секунд занимает задание `test`).

7. Отладка

Ошибки, возникшие при подготовке проекта, в формате «текст ошибки → гипотеза → проверка → решение».

7.1. Строгая сборка падает на ссылках, которых ещё нет

Текст ошибки (`mkdocs build --strict`):

```
1 WARNING - A reference to 'debug.md' is included in the 'nav' configuration, which is not found in the
2 documentation files.
3 WARNING - Doc file 'index.md' contains a link 'debug.md', but the target is not found among documentation
  files.
  Aborted with 7 warnings in strict mode!
```

Гипотеза. В `mkdocs.yml` и в таблице на главной странице уже перечислены разделы, файлы которых я ещё не создала.

Проверка. `ls docs/` показал, что файлов `debug.md`, `conclusion.md` и `links.md` нет; список из 7 предупреждений совпал с этими тремя файлами и ссылками на них.

Решение. Создать недостающие страницы. Режим `--strict` оставлен включённым и в CI: именно так битая ссылка не доходит до публикации.

7.2. Датасет с Google Drive не скачивается автоматически

Текст ошибки. Страница файла `dataset.zip` отдаёт форму входа в аккаунт, а не содержимое файла, поэтому скрипт или робот без авторизации получает HTML вместо архива.

Гипотеза. Ссылка вида `drive.google.com/file/d/<id>/view` — это страница просмотра с требованием входа, а не прямая ссылка на файл.

Проверка. Запрос страницы вернул описание файла (`dataset.zip`) и приглашение войти, без байтов архива.

Решение. Скачать архив вручную из своего аккаунта и распаковать CSV в `data/raw/`. До этого конвейер работает на синтетических демо-данных и честно отмечает это баннером на странице.

7.3. Установка пакета из сети подготовки блокируется

Текст ошибки (`pip install plotly`):

```
1 ProxyError('Cannot connect to proxy.', OSError('Tunnel connection failed: 403 Forbidden'))
2 ERROR: No matching distribution found for plotly
```

Гипотеза. Сетевая политика среды, в которой я готовила проект, не пропускает запросы к индексу пакетов для новых зависимостей; MkDocs и Material при этом были установлены заранее.

Проверка. Повторный запуск с явным указанием прокси дал тот же ответ 403; значит, дело в политике доступа, а не в версии `pip`.

Решение. Не обходить ограничение. Интерактивные графики Plotly для РЗ не требуются, поэтому графики строятся на `matplotlib`, который уже был в окружении. В CI на GitHub такого ограничения нет, и Plotly можно добавить в `requirements.txt`.

7.4. Предупреждение о MkDocs 2.0 в логе сборки

Текст. При каждой сборке тема Material печатает предупреждение, что MkDocs 2.0 сломает плагины и переопределения тем.

Гипотеза. Если в CI поставить зависимости без точных версий, `pip` может подтянуть несовместимую мажорную версию, и сборка сломается без изменений в моём коде.

Проверка. Версии в `requirements.txt` зафиксированы точно: `mkdocs==1.6.1`, `mkdocs-material==9.7.6`; сборка на чистом окружении даёт тот же результат.

Решение. Держать точные версии и обновлять их осознанно, отдельным коммитом.

7.5. Сайт делает внешние запросы к api.github.com

Текст. При проверке в Chromium (учёт всех сетевых запросов страницы) обнаружились два запроса, хотя на странице нет ни одного внешнего скрипта, шрифта или картинки:

```
1 https://api.github.com/repos/777werona-afk/vr-gaze-site/releases/latest
2 https://api.github.com/repos/777werona-afk/vr-gaze-site
```

Гипотеза. Источник — виджет репозитория в шапке темы Material: при заданном `repo_url` он подгружает с GitHub число звёзд и версию последнего релиза. Для хостинга без доступа к внешним сервисам это лишняя зависимость.

Проверка. Убрала `repo_url` и `repo_name` из `mkdocs.yml`, пересобрала сайт и повторила замер: список внешних запросов стал пустым.

Решение. Ссылка на репозиторий осталась в подвале (`extra.social`), а виджет отключён. Страницы загружаются целиком из самого сайта.

7.6. На реальных данных «тремор» выходит в 7–12 градусов

Текст. Первый запуск скрипта на моих 8 сессиях дал в сводной таблице значения, которых в записи быть не может:

```
1 gaze_session_20260916_210658 джиттер до 12.762° после 10.771°
2 gaze_session_20260916_205941 снижение: -214162929767755.2 %
```

Гипотезы. (а) Деление на почти нулевое значение в первой сессии. (б) Ступеньки в сигнале угла, которые полосовой фильтр принимает за колебания. (в) Переход через $\pm 180^\circ$ у горизонтального угла.

Проверка. Распечатала диапазоны углов и шаги между кадрами. В первой сессии вектор взгляда не меняется вовсе, и RMS равен нулю: деление на ноль подтвердилось. Во второй сессии диапазон горизонтального угла оказался от -1129° до 54° , а шагов больше 20° между кадрами нашлось 18 (до 174° за 14 мс). Развёртка угла превращала каждый такой скачок в постоянный сдвиг. Разрывы встречаются парами с интервалом около 37–40 кадров, то есть сигнал на полсекунды «переворачивается» и возвращается.

Решение. Статичные сессии помечаются и не участвуют в сравнении. Шаги больше 30° между кадрами вырезаются, сегменты склеиваются, после чего тремор в полосе 4–12 Гц упал с 7 до 0,2–0,4°. Кроме того, перед этим я добавила чтение файла с BOM (`utf-8-sig`): первый столбец иначе читался как `\uffefftimestamp`.

7.7. Коммит ушёл на GitHub от другого аккаунта

Текст. В истории репозитория у первого коммита `46be2a4` был автор не `777werona-afk`, а другой аккаунт, хотя запуск в Actions значился «pushed by 777werona-afk».

Гипотеза. Автор коммита берётся из `user.name` и `user.email` в настройках Git на компьютере, а не из аккаунта, под которым выполнен `git push`. В глобальных настройках осталась другая учётная запись.

Проверка. `git log -1 --format="%an <%ae>"` показал имя и почту из глобального конфига, а страница пользователя на GitHub подтвердила, что пушит нужный аккаунт.

Решение. Задала имя и почту только для этого репозитория (noreply-адрес вида `ID+777werona-afk@users.noreply.github.com`), пересоздала коммит командой `git commit --amend --reset-author` и заменила его на GitHub через `git push --force-with-lease`. Новый коммит `7977203` создан от `777werona-afk`.

7.8. Намеренно проваленный запуск

Чтобы показать, что режим `--strict` ловит битые ссылки, достаточно добавить в любую страницу ссылку на несуществующий файл:

```
1 [битая ссылка](нет-такой-страницы.md)
```

Сборка завершается ошибкой, job `lint` становится красным, и ни `build`, ни `deploy` не запускаются.

Я проверила это на реальном запуске. В `docs/links.md` я добавила строку - [Демонстрация падения сборки](missing-page.md) и запустила её в `main` (коммит `845bd78`). Запуск №5 завершился ошибкой за 39 секунд: `lint` красный, `build`, `deploy-pages` и `deploy-cloudflare` пропущены, поэтому опубликованный сайт остался прежним.

The screenshot shows a GitHub Actions workflow run for the job 'Build and deploy docs'. The workflow is titled 'Демонстрация: битая ссылка в links.md (сборка должна упасть) #5'. It was triggered via a push 1 minute ago by user 777werona-afk on commit 845bd78 to the main branch. The workflow status is 'Failure' with a total duration of 39s. The workflow steps are: lint (failed, 36s), build (skipped, 0s), deploy-cloudflare (skipped, 0s), and deploy-pages (skipped, 0s). The lint step failed due to a broken link in links.md.

← Build and deploy docs

❌ Демонстрация: битая ссылка в links.md (сборка должна упасть) #5

Summary ▾

Triggered via push 1 minute ago

777werona-afk pushed -> 845bd78 main

Status	Total duration	Artifacts
Failure	39s	—

deploy.yml
on: push

```
graph LR; lint[lint 36s] --> build[build 0s]; build --> deploy-cloudflare[deploy-cloudflare 0s]; build --> deploy-pages[deploy-pages 0s];
```

В логе шага `Build in strict mode` MkDocs прямо называет файл и ссылку:

```
lint
failed 1 minute ago in 36s

16 | MKDocs 2.0, the underlying framework of Material for MkDocs,
17 | will introduce backward-incompatible changes, including:
18 |
19 | x All plugins will stop working - the plugin system has been removed
20 | x All theme overrides will break - the theming system has been rewritten
21 | x No migration path exists - existing projects cannot be upgraded
22 | x Closed contribution model - community members can't report bugs
23 | x Currently unlicensed - unsuitable for production use
24 |
25 | Our full analysis:
26 |
27 | https://squidfunk.github.io/mkdocs-material/blog/2026/02/18/mkdocs-2.0/
28 |
29 | INFO - Cleaning site directory
30 | INFO - Building documentation to directory: /tmp/site-lint
31 | WARNING - Doc file 'links.md' contains a link 'missing-page.md', but the target
32 | is not found among documentation files.
33 |
34 | Aborted with 1 warnings in strict mode!
35 | Error: Process completed with exit code 1.

Post Run actions/setup-python@v5 0s
> Post Run actions/checkout@v4 0s
> Complete job 0s
```

```
1 | WARNING - Doc file 'links.md' contains a link 'missing-page.md', but the target is not found among
2 | documentation files.
3 | Aborted with 1 warnings in strict mode!
  | Error: Process completed with exit code 1.
```

Исправила так: убрала добавленную строку (коммит 83e27e3). Запуск №6 прошёл полностью: `lint`, `build`, `deploy-pages` и `deploy-cloudflare` зелёные.

← Build and deploy docs

✓ Исправление: убрана битая ссылка из links.md #6

Summary ▾

Triggered via push 2 minutes ago

777werona-afk pushed -o- 83e27e3 main

Status	Total duration	Artifacts
Success	<u>1m 52s</u>	<u>2</u>

deploy.yml

on: push



8. Заключение

8.1. Рекомендуемый стек для публикации результатов моей работы

Для публикации результатов экспериментов по записи взгляда в VR я рекомендую связку: **MkDocs с темой Material** как генератор, **Python-скрипт обработки данных**, вызываемый из `Makefile` перед сборкой, **GitHub Actions** (или совместимый GitVerse) для автоматизации, **GitHub Pages** как основную публикацию и **доставку по SSH/rsync** (мой action `rsync-ssh-deploy`) на сервер университета или другого российского хостинга.

Причины выбора:

- Сайт состоит в основном из текста, таблиц и нескольких графиков. Markdown-исходники и готовая тема Material закрывают это без настройки.
- Расчёт отделён от вёрстки: скрипт строит таблицы и графики, MkDocs подключает их как готовые артефакты. Так сборка остаётся быстрой, а результат привязывается к версии кода и данных через метку версии.
- Режим `--strict` и проверка опубликованной страницы после выкладки не дают выложить сломанный сайт.
- Доставка на shell-командах (`wrangler`, `rsync`, `curl`) переносится между платформами почти без изменений, в отличие от действий, специфичных для GitHub.
- По измерениям (раздел 6) выкладка на GitHub Pages занимает около 10 с, по SSH — 16 с, на Cloudflare Pages — около 32 с (из них 10 с — создание проекта, которое нужно только в первый раз). Сборка (в среднем 35 с) и установка зависимостей (около 20 с) дороже любой из выкладок, поэтому ускорять стоит кэш и сборку, а не способ доставки.

8.2. Когда рекомендация меняется

Условие	Что менять
Нужны нумерованные формулы, перекрёстные ссылки на рисунки и BibTeX-библиография, как в статье	перейти на Sphinx с MyST-Parser или Quarto, у них эти возможности встроены
Результаты считаются прямо в ноутбуках и должны исполняться при сборке	Quarto или MyST-NB вместо отдельного скрипта
Вычисления тяжёлые (обучение модели-стабилизатора, видеозаписи сессий)	считать вне CI, а сайт собирать из готовых артефактов; в метке версии хранить хеши данных и модели
Данные содержат записи участников с ОВЗ	не публиковать сырые записи; на сайт выкладывать только агрегированные метрики и получать отдельное согласие участников
Репозиторий и сайт нужно держать в российском контуре	GitVerse, как наиболее близкая по синтаксису к GitHub Actions; публикацию на Pages заменить доставкой по SSH (<code>rsync-ssh-deploy</code>) или S3
Нужно версионирование опубликованных результатов	добавить <code>mile</code> и привязать выкладку версии к git-тегу

8.3. Отклонения от задания и ограничения

- **Отечественный хостинг не задействован.** В задании в качестве второго хоста предполагался отечественный хостинг. Я выбрала Cloudflare Pages по практическим причинам:

- а. Бесплатный тариф покрывает статический сайт без ограничения по сроку, а регистрация не потребовала привязки банковской карты. Российские варианты из обзора T3 (Yandex Object Storage, Timeweb Cloud, Selectel) оплачиваются по тарифам хранения и трафика, то есть для них нужен платёжный аккаунт.
- б. Выкладка делается одной командой `wrangler pages deploy` с токеном API, который хранится в секретах репозитория, и сразу даёт адрес `*.pages.dev` с HTTPS без покупки домена и настройки DNS.
- в. Доступ к Helios ИТМО (вариант из задания) требует учётной записи университета и условий доступа, которых в публичной документации нет (см. раздел 3), поэтому проверить его в рамках работы я не смогла.

Это отклонение от задания, и оно ограничивает выводы: задержки и доступность для пользователей из России на Cloudflare я не измеряла. Вместо отечественного хоста действие `rsync-ssh-deploy` проверено на SSH-сервере, поднятом на раннере. Постоянного сервера у меня нет. - **Action не опубликован в Marketplace** (см. [раздел 5.6](#)): публикация требует принять соглашение разработчика на GitHub и выполняется владельцем аккаунта вручную. - **Измерения** сняты на нескольких запусках на бесплатных раннерах GitHub. Они показывают порядок величин, а не статистически надёжные значения.

8.4. Что не проверено

Результаты на странице РЗ получены на небольшой пилотной записи и описывают один набор данных одного автора. Сравнение групп с ОВЗ и без ОВЗ в этой работе не выполнялось. Время загрузки страниц с точки зрения пользователя из разных регионов не измерялось: измерен только вес страниц и время работы пайплайна.

9. Ссылки и лицензии

9.1. Репозиторий и опубликованные сайты

Что	Адрес
Репозиторий	https://github.com/777werona-afk/vr-gaze-site
Сайт на GitHub Pages	https://777werona-afk.github.io/vr-gaze-site/
Сайт на Cloudflare Pages	https://vr-gaze-site.pages.dev
Репозиторий GitHub Action	https://github.com/777werona-afk/rsync-ssh-deploy
Релиз <code>v1.0.0</code> и теги	https://github.com/777werona-afk/rsync-ssh-deploy/releases/tag/v1.0.0
Workflow, использующий action	<code>deploy.yml</code> , задание <code>deploy-ssh</code>
Страница в GitHub Marketplace	не опубликовано (см. раздел 5.6)
Отечественный хостинг	не развёрнут (см. заключение)

9.2. Лицензии

Контент (`docs/`) — [CC BY 4.0](#), код (`scripts/`, `Makefile`, `mkdocs.yml`, `.github/`) — MIT. Лицензии лежат в корне репозитория: `LICENSE-CONTENT.md` и `LICENSE`.

Сырые записи взгляда участников исследования ни одной из этих лицензий не покрываются: это персональные данные, их публикация требует отдельного согласия.

9.3. Дополнительные материалы

- [Исходные материалы курса на Яндекс.Диске](#)
- [MkDocs](#) и [Material for MkDocs](#)
- [GitHub Docs: создание composite action](#)
- [GitHub Docs: публикация action в Marketplace](#)
- [GitHub Docs: метаданные action \(`action.yml`\)](#)
- [Cloudflare Pages: Wrangler, `pages deploy`](#)
- [Semantic Versioning 2.0.0](#)

Приложение А. Тексты пайплайнов

Листинги подключаются прямо из файлов репозитория при сборке сайта, поэтому не расходятся с реальными файлами. Комментарии написаны в самих файлах.

A.1. Workflow сайта (.github/workflows/deploy.yml)

```
1  name: Build and deploy docs
2
3  # События, запускающие пайплайн:
4  # - push в main          -> lint -> build -> deploy (Pages и Cloudflare)
5  # - pull_request в main  -> lint -> build (деплой пропускается, секреты форков недоступны)
6  # - workflow_dispatch    -> ручной запуск (деплой только если выбрана ветка main)
7  on:
8    push:
9      branches: [main]
10   pull_request:
11       branches: [main]
12   workflow_dispatch:
13
14  # минимальные права по умолчанию; расширяются точно в нужных заданиях
15  permissions:
16    contents: read
17
18  concurrency:
19    group: docs-${{ github.ref }}
20    cancel-in-progress: true
21
22  env:
23    PYTHON_VERSION: "3.11"
24    PAGES_URL: https://777werona-afk.github.io/vr-gaze-site/
25
26  jobs:
27    # ----- 1. lint
28    lint:
29      runs-on: ubuntu-latest
30      steps:
31        - uses: actions/checkout@v4
32
33        - uses: actions/setup-python@v5
34          with:
35            python-version: "${{ env.PYTHON_VERSION }}"
36            cache: pip      # кэш зависимостей по хешу requirements.txt
37
38        - name: Install dependencies
39          run: pip install -r requirements.txt
40
41        - name: Compute results (data -> tables and figures)
42          run: python scripts/analyze.py
43
44        # --strict превращает битые ссылки и предупреждения в ошибку сборки
45        - name: Build in strict mode
46          run: mkdocs build --strict --site-dir /tmp/site-lint
47
48    # ----- 2. build
49    build:
50      needs: lint
51      runs-on: ubuntu-latest
52      steps:
53        - uses: actions/checkout@v4
54
55        - uses: actions/setup-python@v5
56          with:
57            python-version: "${{ env.PYTHON_VERSION }}"
58            cache: pip
59
60        - run: pip install -r requirements.txt
61        - run: python scripts/analyze.py
62
63        # сборка для GitHub Pages
64        - name: Build for GitHub Pages
65          run: mkdocs build --strict --site-dir site
66          env:
67            SITE_URL: "${{ env.PAGES_URL }}"
68
69        # отдельная сборка для Cloudflare (свой базовый URL)
70        - name: Build for Cloudflare Pages
71          run: mkdocs build --strict --site-dir site-hosting
72          env:
```

```

73     SITE_URL: ${vars.HOSTING_SITE_URL || env.PAGES_URL }}
74
75   - uses: actions/upload-pages-artifact@v3
76     with:
77       path: site
78
79   - uses: actions/upload-artifact@v4
80     with:
81       name: site-hosting
82       path: site-hosting
83       retention-days: 7
84
85   # ----- 3a. deploy: GitHub Pages
86   # только с основной ветки; для остальных веток выполняется лишь сборка
87   deploy-pages:
88     needs: build
89     if: github.ref == 'refs/heads/main' && github.event_name != 'pull_request'
90     runs-on: ubuntu-latest
91     permissions:
92       pages: write
93       id-token: write
94     environment:
95       name: github-pages
96       url: ${steps.deployment.outputs.page_url}
97     steps:
98       - id: deployment
99         uses: actions/deploy-pages@v4
100
101       - name: Healthcheck (HTTP 200 + control string)
102         run: |
103           code=$(curl -s -L -o page.html -w '%{http_code}' "${steps.deployment.outputs.page_url}")
104           echo "HTTP $code"
105           test "$code" = "200"
106           grep -q 'Стабилизация взгляда в VR' page.html
107
108   # ----- 3b. deploy: Cloudflare Pages
109   # Нужны секреты репозитория CLOUDFLARE_API_TOKEN и CLOUDFLARE_ACCOUNT_ID и переменная
110   # HOSTING_SITE_URL (адрес вида https://vr-gaze-site.pages.dev). Без секретов задание пропускается.
111   deploy-cloudflare:
112     needs: build
113     if: github.ref == 'refs/heads/main' && github.event_name != 'pull_request'
114     runs-on: ubuntu-latest
115     env:
116       HAS_SECRETS: ${secrets.CLOUDFLARE_API_TOKEN != ''}
117       CLOUDFLARE_API_TOKEN: ${secrets.CLOUDFLARE_API_TOKEN}
118       CLOUDFLARE_ACCOUNT_ID: ${secrets.CLOUDFLARE_ACCOUNT_ID}
119       PROJECT: vr-gaze-site
120     steps:
121       - uses: actions/download-artifact@v4
122         with:
123           name: site-hosting
124           path: site-hosting
125
126       # проект создаётся один раз; если он уже есть, команда завершается ошибкой, и это нормально
127       - name: Create Pages project (once)
128         if: env.HAS_SECRETS == 'true'
129         run: npx --yes wrangler@4 pages project create "$PROJECT" --production-branch main || true
130
131       - name: Deploy to Cloudflare Pages
132         if: env.HAS_SECRETS == 'true'
133         run: npx --yes wrangler@4 pages deploy site-hosting --project-name "$PROJECT" --branch main
134
135       - name: Healthcheck (HTTP 200 + control string)
136         if: env.HAS_SECRETS == 'true' && vars.HOSTING_SITE_URL != ''
137         run: |
138           sleep 10
139           code=$(curl -s -L -o page.html -w '%{http_code}' "${vars.HOSTING_SITE_URL}")
140           echo "HTTP $code"
141           test "$code" = "200"
142           grep -q 'Стабилизация взгляда в VR' page.html
143
144       - name: Skipped (no Cloudflare secrets)
145         if: env.HAS_SECRETS != 'true'
146         run: echo "Секреты Cloudflare не заданы – выкладка пропущена."
147

```

```

148 # ----- 3с. deploy: по SSH собственным
149 action
150 # Используется action https://github.com/777werona-afk/rsync-ssh-deploy (ver v1).
151 # Постоянного SSH-сервера у проекта нет, поэтому job выкладывает сайт на sshd, поднятый на самом
152 раннере,
153 # и проверяет результат. Для реального сервера достаточно заменить host, user, path и ключ на секреты.
154 deploy-ssh:
155   needs: build
156   if: github.ref == 'refs/heads/main' && github.event_name != 'pull_request'
157   runs-on: ubuntu-latest
158   steps:
159     - uses: actions/download-artifact@v4
160       with:
161         name: site-hosting
162         path: site-hosting
163
164     - name: Start local sshd and create a key
165       id: ssh
166       run: |
167         sudo apt-get update -qq && sudo apt-get install -y -qq openssh-server rsync
168         sudo service ssh start
169         ssh-keygen -t ed25519 -N '' -f "$RUNNER_TEMP/id_site" -q
170         mkdir -p ~/.ssh && chmod 700 ~/.ssh
171         cat "$RUNNER_TEMP/id_site.pub" >> ~/.ssh/authorized_keys
172         chmod 600 ~/.ssh/authorized_keys
173         echo "user=$(whoami)" >> "$GITHUB_OUTPUT"
174         {
175           echo 'key<<EOF_KEY'
176           cat "$RUNNER_TEMP/id_site"
177           echo 'EOF_KEY'
178         } >> "$GITHUB_OUTPUT"
179
180     - name: Deploy over SSH with rsync-ssh-deploy
181       id: deploy
182       uses: 777werona-afk/rsync-ssh-deploy@v1
183       with:
184         host: localhost
185         user: ${ steps.ssh.outputs.user }
186         path: /tmp/site-ssh
187         source: site-hosting
188         key: ${ steps.ssh.outputs.key }
189         delete: 'true'
190
191     - name: Check the deployed copy
192       run: |
193         echo "Передано файлов: ${ steps.deploy.outputs.files_transferred }"
194         test -f /tmp/site-ssh/index.html
195         grep -q 'Стабилизация взгляда в VR' /tmp/site-ssh/index.html

```

A.2. Описание action (action.yml)

Файл из репозитория `rsync-ssh-deploy`, версия `v1.0.0`.

```

1 name: 'Rsync SSH Deploy'
2 description: 'Выкладывает каталог (например, собранный статический сайт) на сервер по SSH с помощью
3 rsync; поддерживает удаление лишних файлов.'
4 author: 'Marina Kozhevyatkina'
5
6 branding:
7   icon: 'upload-cloud'
8   color: 'blue'
9
10 inputs:
11   host:
12     description: 'Имя или адрес сервера, например helios.example.org.'
13     required: true
14   user:
15     description: 'Имя пользователя на сервере.'
16     required: true
17   path:
18     description: 'Каталог назначения на сервере (абсолютный или относительно домашнего каталога).'
19     required: true
20   source:
21     description: 'Исходный каталог в рабочей папке задания.'
22     required: false
23     default: 'site'
24   key:
25     description: 'Закрытый SSH-ключ (передавайте через secrets, не в открытом виде).'
26     required: true
27   delete:
28     description: 'Если "true", на сервере удаляются файлы, которых нет в исходном каталоге (флаг rsync -
29 -delete).'
30     required: false
31     default: 'false'
32   port:
33     description: 'Порт SSH.'
34     required: false
35     default: '22'
36   known_hosts:
37     description: 'Строки known_hosts для проверки сервера. Если пусто, отпечаток запрашивается через
38 ssh-keyscan при первом подключении (менее безопасно).'
39     required: false
40     default: ''
41   dry_run:
42     description: 'Если "true", rsync только показывает, что будет сделано (--dry-run).'
43     required: false
44     default: 'false'
45
46 outputs:
47   destination:
48     description: 'Итоговое назначение в формате user@host:path.'
49     value: '{{ steps.sync.outputs.destination }}'
50   files_transferred:
51     description: 'Сколько файлов передано (по статистике rsync).'
52     value: '{{ steps.sync.outputs.files_transferred }}'
53   files_deleted:
54     description: 'Сколько файлов удалено на сервере (по статистике rsync; 0, если delete не включён).'
55     value: '{{ steps.sync.outputs.files_deleted }}'
56
57 runs:
58   using: 'composite'
59   steps:
60     - name: Check inputs
61       shell: bash
62       env:
63         IN_HOST: '{{ inputs.host }}'
64         IN_USER: '{{ inputs.user }}'
65         IN_PATH: '{{ inputs.path }}'
66         IN_SOURCE: '{{ inputs.source }}'
67         IN_DELETE: '{{ inputs.delete }}'
68       run: |
69         # входные данные проверяются до подключения к серверу
70         for v in IN_HOST IN_USER IN_PATH IN_SOURCE; do
71           if [ -z "${!v}" ]; then echo "::error::Параметр ${v#IN_} не задан"; exit 1; fi
72         done
73         if [ ! -d "$IN_SOURCE" ]; then
74           echo "::error::Исходный каталог '$IN_SOURCE' не найден"; exit 1
75         fi

```

```

76     if [ -z "$(ls -A "$IN_SOURCE")" ]; then
77         echo "::error::Исходный каталог '$IN_SOURCE' пуст: выкладка с удалением стёрла бы сайт на
78 сервере"; exit 1
79     fi
80     # защита от опасных путей при удалении
81     if [ "$IN_DELETE" = "true" ]; then
82         case "$IN_PATH" in
83             /|.|-|~|/|") echo "::error::Путь '$IN_PATH' слишком опасен для режима delete"; exit 1;;
84             esac
85         fi
86
87     - name: Prepare SSH key
88     shell: bash
89     env:
90         IN_KEY: ${inputs.key}
91         IN_HOST: ${inputs.host}
92         IN_PORT: ${inputs.port}
93         IN_KNOWN_HOSTS: ${inputs.known_hosts}
94     run: |
95         # ключ и known_hosts лежат во временной папке раннера и удаляются в последнем шаге
96         dir="$RUNNER_TEMP/rsync-ssh-deploy"
97         mkdir -p "$dir" && chmod 700 "$dir"
98         printf '%s\n' "$IN_KEY" > "$dir/key"
99         chmod 600 "$dir/key"
100         if [ -n "$IN_KNOWN_HOSTS" ]; then
101             printf '%s\n' "$IN_KNOWN_HOSTS" > "$dir/known_hosts"
102         else
103             echo "::warning::known_hosts не задан: отпечаток сервера берётся через ssh-keyscan и не
104 проверяется заранее"
105             ssh-keyscan -p "$IN_PORT" "$IN_HOST" > "$dir/known_hosts" 2>/dev/null
106         fi
107         echo "RSD_DIR=$dir" >> "$GITHUB_ENV"
108
109     - name: Sync with rsync
110     id: sync
111     shell: bash
112     env:
113         IN_HOST: ${inputs.host}
114         IN_USER: ${inputs.user}
115         IN_PATH: ${inputs.path}
116         IN_SOURCE: ${inputs.source}
117         IN_DELETE: ${inputs.delete}
118         IN_PORT: ${inputs.port}
119         IN_DRY: ${inputs.dry_run}
120     run: |
121         args=(-az --stats)
122         [ "$IN_DELETE" = "true" ] && args+=(-delete)
123         [ "$IN_DRY" = "true" ] && args+=(-dry-run)
124         ssh_cmd="ssh -i $RSD_DIR/key -p $IN_PORT -o IdentitiesOnly=yes -o
125 UserKnownHostsFile=$RSD_DIR/known_hosts -o StrictHostKeyChecking=yes"
126         dest="$IN_USER@$IN_HOST:$IN_PATH"
127         # слэш после источника: копируется содержимое каталога, а не сам каталог
128         rsync "${args[@]}" -e "$ssh_cmd" "${IN_SOURCE%}/" "${dest%}/" | tee "$RSD_DIR/stats.txt"
129         num() { grep -E "$1" "$RSD_DIR/stats.txt" | head -n1 | awk -F: '{gsub(/[ ,/,"",]$2); print $2}';
130     }
131     {
132         echo "destination=$dest"
133         echo "files_transferred=$(num 'Number of (regular )?files transferred')"
134         echo "files_deleted=$(num 'Number of deleted files')"
135     } >> "$GITHUB_OUTPUT"
136
137     - name: Remove key
138     if: always()
139     shell: bash
140     run: |
141         # ключ не остаётся на раннере даже при ошибке rsync
142         [ -n "$RSD_DIR:-" ] && rm -rf "$RSD_DIR"
143         true

```

A.3. Тестовый workflow action (`.github/workflows/test.yml`)

```
1  name: Test action
2
3  # Проверка action на самом раннере: поднимаем локальный sshd и выкладываем каталог на localhost.
4  on:
5    push:
6      branches: [main]
7    pull_request:
8    workflow_dispatch:
9
10 permissions:
11   contents: read
12
13 jobs:
14   test:
15     runs-on: ubuntu-latest
16     steps:
17       - uses: actions/checkout@v4
18
19       - name: Start local sshd and create a key
20         id: ssh
21         run: |
22           sudo apt-get update -qq && sudo apt-get install -y -qq openssh-server rsync
23           sudo service ssh start
24           ssh-keygen -t ed25519 -N '' -f "$RUNNER_TEMP/id_test" -q
25           mkdir -p ~/.ssh && chmod 700 ~/.ssh
26           cat "$RUNNER_TEMP/id_test.pub" >> ~/.ssh/authorized_keys
27           chmod 600 ~/.ssh/authorized_keys
28           echo "user=$(whoami)" >> "$GITHUB_OUTPUT"
29           {
30             echo 'key<<EOF_KEY'
31             cat "$RUNNER_TEMP/id_test"
32             echo 'EOF_KEY'
33           } >> "$GITHUB_OUTPUT"
34
35       - name: Prepare source directory
36         run: |
37           mkdir -p src/sub
38           echo "one" > src/index.html
39           echo "two" > src/sub/page.html
40
41       - name: First deploy
42         id: first
43         uses: ./
44         with:
45           host: localhost
46           user: ${ steps.ssh.outputs.user }
47           path: /tmp/deploy-target
48           source: src
49           key: ${ steps.ssh.outputs.key }
50           delete: 'true'
51
52       - name: Create a stale file on the server and remove a file from the source
53         run: |
54           echo "stale" > /tmp/deploy-target/stale.html
55           rm src/sub/page.html
56
57       - name: Second deploy (must delete stale files)
58         id: second
59         uses: ./
60         with:
61           host: localhost
62           user: ${ steps.ssh.outputs.user }
63           path: /tmp/deploy-target
64           source: src
65           key: ${ steps.ssh.outputs.key }
66           delete: 'true'
67
68       - name: Check result and outputs
69         run: |
70           echo "first: ${ steps.first.outputs.files_transferred } transferred"
71           echo "second: ${ steps.second.outputs.files_deleted } deleted"
72           test -f /tmp/deploy-target/index.html
```

```

73     test ! -e /tmp/deploy-target/stale.html
74     test ! -e /tmp/deploy-target/sub/page.html
75     test "${{ steps.second.outputs.destination }}" = "${{ steps.ssh.outputs.user
76 }}@localhost:/tmp/deploy-target"
77
78     - name: Create an empty source directory
79       run: mkdir empty
80
81     - name: Empty source must be rejected
82       id: empty
83       continue-on-error: true
84       uses: ./
85       with:
86         host: localhost
87         user: "${{ steps.ssh.outputs.user }}"
88         path: /tmp/deploy-target
89         source: empty
90         key: "${{ steps.ssh.outputs.key }}"
91         delete: 'true'
92
93     - name: Check that the empty source failed
94       run: test "${{ steps.empty.outcome }}" = "failure"

```

A.4. Перестановка мажорного тега (`.github/workflows/release.yml`)

```

1  name: Move major tag
2
3  # При публикации тега vX.Y.Z подвижный тег vX переставляется на тот же коммит,
4  # чтобы пользователи могли писать uses: owner/геро@v1 и получать исправления.
5  on:
6    push:
7      tags:
8        - 'v[0-9]+.[0-9]+.[0-9]+'
9
10 permissions:
11   contents: write
12
13 jobs:
14   major-tag:
15     runs-on: ubuntu-latest
16     steps:
17       - uses: actions/checkout@v4
18       - name: Move vX tag
19         run: |
20           major="${GITHUB_REF_NAME%.*}"
21           git tag -f "$major" "$GITHUB_SHA"
22           git push -f origin "$major"

```